

Final Evolution Lab — AWS Budget Optimization Guide

Target Budget: \$400/month

Last Updated: April 2026

Region: us-west-2 (Oregon) — cheapest for GPU instances

Important: Cloud Costs Are Ongoing

AWS charges are billed continuously while resources are running. Unlike a one-time purchase, cloud infrastructure costs recur every month. If you forget to shut down instances, you will be billed. Always:

- Set up **billing alerts** in AWS Console → Billing → Budgets
 - Use **scheduled scaling** to turn off instances when not needed
 - Review your AWS bill weekly during the first month
 - Set a **hard budget alarm** at \$400 to get notified before overspending
-



Cost Breakdown by Component

Core Infrastructure Costs (us-west-2, April 2026 pricing)

Component	On-Demand (24/7)	Spot (24/7)	Spot (14h/day)	Notes
g4dn.xlarge GPU (1 instance)	\$378/mo	\$113-151/mo	\$66-88/mo	NVIDIA T4, 4 vCPU, 16 GB
g5.xlarge GPU (1 instance)	\$760/mo	\$228-380/mo	\$133-222/mo	NVIDIA A10G, 4 vCPU, 16 GB
Application Load Balancer	\$22/mo	\$22/mo	\$22/mo	Fixed cost + LCU charges
S3 Storage (50 GB builds)	\$1-5/mo	\$1-5/mo	\$1-5/mo	Minimal at this scale
Data Transfer (100 GB out)	\$9/mo	\$9/mo	\$9/mo	First 100 GB free tier
CloudWatch (logs + metrics)	\$5-10/mo	\$5-10/mo	\$5-10/mo	Basic monitoring
Elastic IP (idle)	\$3.60/mo	\$3.60/mo	\$3.60/mo	Free when attached to running instance
TURN Server (c5.xlarge)	\$122/mo	-	-	Optional — see below
TURN Server (t3.small)	\$15/mo	-	-	Budget alternative

Deployment Scenarios

● Scenario A: Ultra-Budget (\$150-210/month) — RECOMMENDED

- ✓ 1× g4dn.xlarge SPOT instance
- ✓ Scheduled scaling: 14 hours/day (7 AM - 9 PM PST)
- ✓ No TURN server
- ✓ Minimal CloudWatch

Item	Monthly Cost
GPU Spot Instance (14h × 30d × ~\$0.16/hr)	\$67-96
Application Load Balancer	\$22
S3 + Data Transfer	\$5-15
CloudWatch	\$5-10
TOTAL	\$99-143

 **Best for:** Development, demos, testing, small user base

Scenario B: Standard Production (\$200-300/month)

-  1× g4dn.xlarge SPOT instance
-  Scheduled scaling: 16 hours/day (6 AM - 10 PM PST)
-  TURN server (t3.small)
-  Full CloudWatch monitoring

Item	Monthly Cost
GPU Spot Instance (16h × 30d × ~\$0.16/hr)	\$77-110
Application Load Balancer	\$22
TURN Server (t3.small, 24/7)	\$15
S3 + Data Transfer	\$10-20
CloudWatch + Alarms	\$10-15
TOTAL	\$134-182

 **Best for:** Small production deployment, up to ~50 concurrent users

Scenario C: High Availability (\$350-500/month)

-  1× g4dn.xlarge ON-DEMAND (always-on baseline)
-  Auto-scale to 2× with SPOT instances
-  TURN server (t3.small)
-  Full monitoring + alerts

Item	Monthly Cost
GPU On-Demand (1× baseline, 24/7)	\$378
Spot Instance (scale-up only, ~4h/day avg)	\$19
Application Load Balancer	\$22
TURN Server (t3.small)	\$15
S3 + Data + CloudWatch	\$20-30
TOTAL	\$454-464

⚠ **Exceeds \$400 budget** — Use Scenario A or B instead, or see Reserved Instances below

Cost-Saving Strategies

1. Spot Instances (Save 60-70%)

Spot instances use spare AWS capacity at massive discounts. For `g4dn.xlarge` :

- **On-Demand:** \$0.526/hr (~\$378/month)
- **Spot:** \$0.13-0.21/hr (~\$95-151/month)
- **Savings: \$190-280/month per instance!**

Trade-off: AWS can reclaim spot instances with 2-minute warning. Mitigation:

- Use multiple instance types (`g4dn.xlarge` + `g4dn.2xlarge`) for better availability
- Use `capacity-optimized` allocation strategy
- UE5 Pixel Streaming sessions are stateless — reconnection is seamless
- Users get a brief interruption (reconnect within ~2 min)

Enable in Terraform:

```
# In terraform.tfvars
use_spot_instances      = true
spot_max_price          = "0.25"
spot_instance_types     = ["g4dn.xlarge", "g4dn.2xlarge"]
spot_allocation_strategy = "capacity-optimized"
```

2. Scheduled Scaling (Save 40-60%)

Run instances only during business hours. Why pay for GPU when nobody is using it?

```
# In terraform.tfvars
enable_scheduled_scaling = true
schedule_scale_down_cron = "0 6 * * *" # 10 PM PST → shut down
schedule_scale_up_cron   = "0 15 * * *" # 7 AM PST → start up
off_hours_min_capacity    = 0           # 0 = fully off at night
```

Cost impact for `g4dn.xlarge` spot:

| Schedule | Hours/Day | Monthly Cost |

|-----|:-----|:-----|
 | 24/7 (always on) | 24h | \$113-151 |
 | Business hours (14h) | 14h | \$66-88 |
 | Peak only (8h) | 8h | \$38-50 |
 | Weekdays only (14h × 5d) | ~10h avg | \$47-63 |

3. Smaller Instance Types

Instance	GPU	vCPU	RAM	On-Demand/hr	Spot/hr	Best For
g4dn.xlarge	T4 16GB	4	16 GB	\$0.526	\$0.13-0.21	✅ Best value for Pixel Streaming
g4dn.2xlarge	T4 16GB	8	32 GB	\$0.752	\$0.19-0.30	Multiple streams per instance
g5.xlarge	A10G 24GB	4	16 GB	\$1.006	\$0.30-0.50	Higher quality rendering
g5.2xlarge	A10G 24GB	8	32 GB	\$1.212	\$0.36-0.61	Premium quality

Recommendation: `g4dn.xlarge` is the sweet spot for \$400 budget. The T4 GPU handles UE5 Pixel Streaming at 1080p 30-60fps with good quality.

4. Reserved Instances (Save 30-40% on long-term)

If you commit to running 24/7 for 1-3 years:

Commitment	g4dn.xlarge Monthly	Savings vs On-Demand
On-Demand	\$378/mo	—
1-Year Reserved (No Upfront)	\$241/mo	36%
1-Year Reserved (All Upfront)	\$220/mo	42%
3-Year Reserved (All Upfront)	\$142/mo	62%

⚠️ **Not recommended at this stage.** Start with Spot instances. Only consider Reserved after you've validated your usage patterns for 2-3 months.

5. Disable TURN Server

Most users on modern networks (home WiFi, 4G/5G) can connect directly via STUN. TURN is only needed for users behind strict corporate firewalls.

```
turn_server_enabled = false # Saves $15-120/month
```

If some users need TURN, use a smaller instance:

```
turn_server_enabled = true
turn_instance_type = "t3.small" # $15/mo vs $122/mo for c5.xlarge
```

Comparison Table: On-Demand vs Spot vs Reserved

Feature	On-Demand	Spot	Reserved (1yr)
g4dn.xlarge \$/hr	\$0.526	\$0.13-0.21	\$0.329
Monthly (24/7)	\$378	\$95-151	\$237
Monthly (14h/day)	\$221	\$55-88	\$138
Availability	✅ Guaranteed	⚠️ Can be interrupted	✅ Guaranteed
Commitment	None	None	1 year
Best For	Production HA	Dev/Demo/Budget	Steady 24/7 usage
Interruption Risk	None	5-10% chance/month	None
Budget Fit (\$400)	❌ Tight (1 inst)	✅ Comfortable	⚠️ Possible



Step-by-Step: Deploy Within \$400 Budget

Step 1: Copy the Budget-Optimized Config

```
cd deployment/aws/
cp terraform.tfvars.example terraform.tfvars
```

Step 2: Set Your AWS Credentials

```
export AWS_ACCESS_KEY_ID="your-access-key"
export AWS_SECRET_ACCESS_KEY="your-secret-key"

# Or configure via AWS CLI
aws configure
```

Step 3: Set Your Key Pair

Ensure the `FinalEvolutionLab` key pair exists in your AWS account:

```
aws ec2 describe-key-pairs --key-names FinalEvolutionLab --region us-west-2
```

If not, import it:

```
aws ec2 import-key-pair \
  --key-name FinalEvolutionLab \
  --public-key-material fileb:// ~/.ssh/FinalEvolutionLab.pub \
  --region us-west-2
```

Step 4: Initialize and Deploy

```
terraform init
terraform plan -out=tfplan
# Review the plan carefully!
terraform apply tfplan
```

Step 5: Set Up Billing Alerts

Critical! Set up a budget in AWS Console:

1. Go to **AWS Console → Billing → Budgets**
2. Create a **Cost Budget** for \$400/month
3. Set alerts at **50%** (\$200), **80%** (\$320), and **100%** (\$400)
4. Add your email for notifications

Or via CLI:

```
aws budgets create-budget \
  --account-id $(aws sts get-caller-identity --query Account --output text) \
  --budget '{
    "BudgetName": "FEL-Monthly-Budget",
    "BudgetLimit": {"Amount": "400", "Unit": "USD"},
    "TimeUnit": "MONTHLY",
    "BudgetType": "COST"
  }' \
  --notifications-with-subscribers '[{
    "Notification": {
      "NotificationType": "ACTUAL",
      "ComparisonOperator": "GREATER_THAN",
      "Threshold": 80,
      "ThresholdType": "PERCENTAGE"
    },
    "Subscribers": [{
      "SubscriptionType": "EMAIL",
      "Address": "deploy@finalevolutiongroup.com"
    }]
  }]'
```

Step 6: Verify Spot Instance Pricing

Check current spot prices before deploying:

```
aws ec2 describe-spot-price-history \
  --instance-types g4dn.xlarge \
  --product-descriptions "Linux/UNIX" \
  --region us-west-2 \
  --start-time $(date -u +%Y-%m-%dT%H:%M:%S) \
  --query 'SpotPriceHistory[*].{AZ:AvailabilityZone,Price:SpotPrice}' \
  --output table
```



Monthly Cost Calculator

Use this formula to estimate your monthly cost:

```
GPU Cost = (spot_price_per_hr)  (hours_per_day)  (days_per_month)
ALB Cost = $22 (fixed)
Storage = $5-15 (varies with usage)
Monitoring = $5-10
TURN = $0 (disabled) or $15 (t3.small)

TOTAL = GPU + ALB + Storage + Monitoring + TURN
```


Quick Reference Table

Hours/Day	Spot g4dn.xlarge	+ ALB + Extras	Total/Month
8h	\$38-50	\$32-47	\$70-97
12h	\$56-76	\$32-47	\$88-123
14h	\$66-88	\$32-47	\$98-135
16h	\$77-101	\$32-47	\$109-148
20h	\$95-126	\$32-47	\$127-173
24h (always on)	\$113-151	\$32-47	\$145-198

All scenarios fit within the \$400 budget! Even running 24/7 with spot instances leaves room for extras.

Handling Spot Interruptions

When AWS reclaims a spot instance, your users will briefly lose their streaming session. Here's how to handle it gracefully:

1. **Auto Scaling Group** automatically launches a replacement instance
2. **UE5 Pixel Streaming** clients auto-reconnect via the signaling server
3. **ALB health checks** route new connections to healthy instances
4. **User experience:** ~2-5 minute interruption, then auto-reconnect

Best Practices for Spot Reliability:

- Use **multiple instance types** in `spot_instance_types`
- Use **multiple Availability Zones** (already configured in VPC)
- Use **capacity-optimized** allocation strategy
- Consider **1 on-demand base** + spot for scale (costs more but zero-interruption baseline)

```
# For higher reliability with spot:
on_demand_base_capacity      = 1    # 1 always-on instance
on_demand_percentage_above_base = 0  # Scale with spot
```

Emergency Cost Reduction

If you're approaching your budget limit:

Immediate Actions (minutes)

```
# Scale to 0 instances immediately
aws autoscaling set-desired-capacity \
  --auto-scaling-group-name final-evolution-lab-production-gpu-asg \
  --desired-capacity 0 \
  --region us-west-2

# Or destroy everything
cd deployment/aws/
terraform destroy
```

Reduce Hours

```
# Run only 8 hours on weekdays
schedule_scale_down_cron = "0 1 * * *" # 5 PM PST
schedule_scale_up_cron   = "0 17 * * 1-5" # 9 AM PST, Mon-Fri only
```

Switch to Smaller Volume

```
root_volume_size = 50 # Minimum for UE5 builds
```



Additional Resources

- [AWS Spot Instance Advisor](https://aws.amazon.com/ec2/spot/instance-advisor/) (https://aws.amazon.com/ec2/spot/instance-advisor/) — Check interruption rates
 - [AWS Pricing Calculator](https://calculator.aws/) (https://calculator.aws/) — Detailed cost estimation
 - [EC2 Spot Best Practices](https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-best-practices.html) (https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-best-practices.html)
 - [GPU Instance Comparison](https://instances.vantage.sh/?filter=g4dn,g5) (https://instances.vantage.sh/?filter=g4dn,g5) — Compare all GPU instances
-

This guide is part of the Final Evolution Lab deployment documentation.

For deployment instructions, see [DEPLOYMENT_GUIDE.md](#) (../../DEPLOYMENT_GUIDE.md)